

Designed by Remus Teodorescu 2026

LECTURE 11.1

Vision-Based Navigation

Remus Teodorescu

With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

CAMERA · POLICY · LATENCY



The JetRacer Pro: a camera, a Jetson Nano and a steering servo, and the first lecture in Part 2 with no differential equation. A network steers the car from one frame at a time, so between two decisions the car drives blind - the lecture is about that loop: the policy, the data it must be shown deliberately, and the deployment - fewer bits, a compiled engine - that sets its rate.

Learning Objectives

SECTIONS, IN ORDER

1. **The car** — the platform, and the loop that drives blind
2. **The policy** — steering from a frame, data collected deliberately
3. **Deployment** — fewer bits, and a model compiled for the device

UDL carries the convolutional network and the transfer; the hardware figures are the verified ones of the course's own specification sheet. The example is Ex_11.1's line follower: a study notebook on drawn frames, where the true answer is known, and an on-car notebook, where the real loop rate is measured.

READ, AND WHAT EACH ANSWERS

UDL ch. 10	what does a CNN see in a frame?
UDL ch. 9	how is a backbone reused?
PyTorch	how are weights deployed?

Recap of L10.2 Solid Oxide Cells

1 – 2 · A NETWORK FOR THE FUEL CHANNEL

1 **How can a network learn the steam fraction along the channel of the cell for every cell voltage?**

The network takes the position along the channel and the cell voltage, and is trained over 1.10 to 1.40 V until the species equation holds at points along the channel, with the local current set by the gap between the cell voltage and the local Nernst potential. The steam fraction at the inlet is built into the form of the solution, and the zero slope at the outlet is held by a loss term. After one training, any voltage in the range is read off, not solved again. *Liu ch. 2.2; Ex_10.2 section 5*

2 **Why does the local current make the fuel channel harder to solve, and how is the network's answer checked?**

The flux at the electrode is no longer given: the current at each point follows the local steam fraction, so the equation is nonlinear in it and needs a condition at the outlet, and it can no longer be marched from the inlet. The network takes the nonlinear term into its residual with no change of method. Its answer is scored against a boundary-value solver at a tight tolerance: at 1.29 V within 0.0005 of the steam fraction, where finite volumes on 50 cells are within 0.0042. *Ex_10.2 sections 3 – 5*

3 – 4 · THE OPTIMAL VOLTAGE

3 **How can the optimal cell voltage be found with a network, and what does the gradient save?**

The worth of an hour, the hydrogen made at its price less the electricity used at its price, is computed from the network's steam fraction and current. With the voltage as an input, automatic differentiation gives the derivative of the worth with respect to it, and gradient ascent finds 1.30 V in 200 steps and 0.2 s; a sweep of 61 runs of the reference solver finds the same voltage in 4 s. The prices are assumptions, so the voltage found is conditional on them. *Liu ch. 5; Ex_10.2 section 6*

4 **Why does a network help to optimise the operation of the cell over its lifetime, and what should not be claimed from it?**

Over a lifetime the decision is a trajectory of the voltage and the temperature, because what produces more now ages the cell faster. A solver gives the worth, but its gradient costs one solve per variable per step, or an adjoint built by hand; a trained network gives the gradient with respect to every input by automatic differentiation. Degradation rates are uncertain by an order of magnitude, so a predicted lifetime ranks schedules; it is not a number to promise. *Liu ch. 5*

The JetRacer Pro

JetRacer Pro — a quarter-scale car that computes on board: a Jetson Nano runs the network, a wide camera is its only sensor, a servo steers and a brushed motor drives. Modest hardware on purpose: the constraints are visible, and a workflow that holds here holds anywhere.

TWO TRAPS BEFORE THE FIRST RUN

power mode — the Nano ships slow: set the 10 W mode, and check

the camera mount — move it a centimetre, and every image lies: recollect



the kit, as it arrives

THE VERIFIED SPECIFICATION

compute	Jetson Nano 4 GB, 128-core Maxwell
camera	IMX219-160: 8 MP, 160° field of view
steering	Ackermann, 6 kg·cm servo
drive	4WD, RC380 brushed motor
pack	8.4 V, four 18650 cells, 2S2P
encoder	none

docs/hardware/jetracer-pro.md is the course's checked sheet - quote it, not memory: the early drafts carried the plain JetRacer's pack and drivetrain. No encoder is why L11.2's speed trace is the open problem it is.

The Control Loop

Blind distance — what the car covers between two decisions, driving the last command with no new information: its speed over the decisions per second. A model slightly more accurate and half as fast is usually a worse car, so accuracy and loop rate are judged together.

THE ONE NUMBER OF THE LOOP

$$d = \frac{v}{f}$$

d the blind distance, v the car's speed, f decisions per second : the whole loop, not the network alone

ONE PASS OF THE LOOP



meanwhile the car drives the last command: $d = v/f$ of track, blind

TALK · 2 MIN How would you explain, in your own words, why a slightly more accurate but slower model can be a worse car? Then, in the chain above, which stage would you time first, and why?

THE SIMULATED LAP OF EX_11.1

blind up to 10 cm	holds the line within 2 cm
at 13 cm	3 cm off the line
at 17 cm	leaves the course
1 m/s at 10 Hz	drives like 2 m/s at 20 Hz
so time	the whole loop, not the net

Capture, preprocessing, inference, actuation: every stage costs time, and only one is the network.

Steering From a Frame

Vision policy — a network that maps one camera frame to one action. The task lives in the head and the loss, not in the body: two logits make an obstacle stopper, one number between −1 and +1 makes a steerer, and the continuous output is what steers smoothly.

EX_11.1: WHERE IS THE LINE, A LITTLE AHEAD

the frames	48 × 64 grey pixels, drawn, so the truth is known
the detector	bright pixels in a band of rows
the network	a small CNN, 55 233 weights
what breaks one	a patch of glare on the floor

MEAN ERROR ON 1000 HELD-OUT FRAMES

	detector	network
along the line	0.024	0.006
beside the line	0.026	0.008
with glare	0.369	0.009
one frame	0.04 ms	0.19 ms
training	none	3 s

The detector at 12 × 16 pixels is the coarsest that meets the agreed 0.03 along the line - and glare breaks it twelvefold, while the network, shown glare in training, shrugs it off. The network row is the deliberately trained one; the next slide is why.

Collect the Data Deliberately

Deliberate collection — choosing the frames the policy will be trained on. The network learns only what the pictures show: the floor, the mount and the light it will drive in, both outcomes in balance, and on purpose the bad positions the car reaches when it is already going wrong.

THE RULES, AND THE ONE THAT IS NOT OPTIONAL

where it will drive	same floor, same light, same mount
balance	both outcomes, a few hundred each
vary the rest	positions, headings, distances
recovery states	half off the line, at an angle: label the way back

The last rule is the compounding error of imitation arriving in practice: a policy shown only perfect driving has never seen a mistake and cannot leave one.

THE SAME NETWORK, TRAINED TWO WAYS

mean error	along only	deliberate
along the line	0.002	0.006
beside the line	0.136	0.008
with glare	0.149	0.009

Trained only along the line the network is seventeen times worse exactly where a drifting car needs it. Most failures on the floor are collection errors, not architecture errors.

Transfer Learning

Transfer learning — reusing a network trained on one task as the start of another: pre-training, on hardware you can hold. A ResNet-18 trained on ImageNet already detects edges, textures and shapes - none of it specific to cats - so only the head is new, and a few hundred frames are enough.

THE ON-CAR RECIPE, IN FOUR LINES

BACKBONE KEPT, HEAD REPLACED

```
net = torchvision.models.resnet18(pretrained=True)   # the backbone, kept
net.fc = torch.nn.Linear(512, 1)                     # the new head: one steering number
x = normalize(frame, IMAGENET_MEAN, IMAGENET_STD)    # the backbone's own statistics
opt = torch.optim.Adam(net.parameters(), lr=1e-4)    # fine-tune it all, gently
```

The study notebook's small CNN trains from nothing, because its frames are drawn in thousands; on the car a few hundred real frames meet a reused backbone.

THE SILENT ERROR

forget it	and the model still trains
the loss	still falls
the accuracy	mediocre, for no visible reason
so	it is the first thing to check

The backbone expects its own mean and standard deviation, channel by channel; fed raw pixels it sees every frame through the wrong glass.

Fewer Bits

Quantisation — storing each weight in b bits instead of 32: the range of the weights is cut into 2^b steps and every weight is rounded to its step. Smaller weights move faster through a small GPU's memory, and the question is how few bits the policy survives.

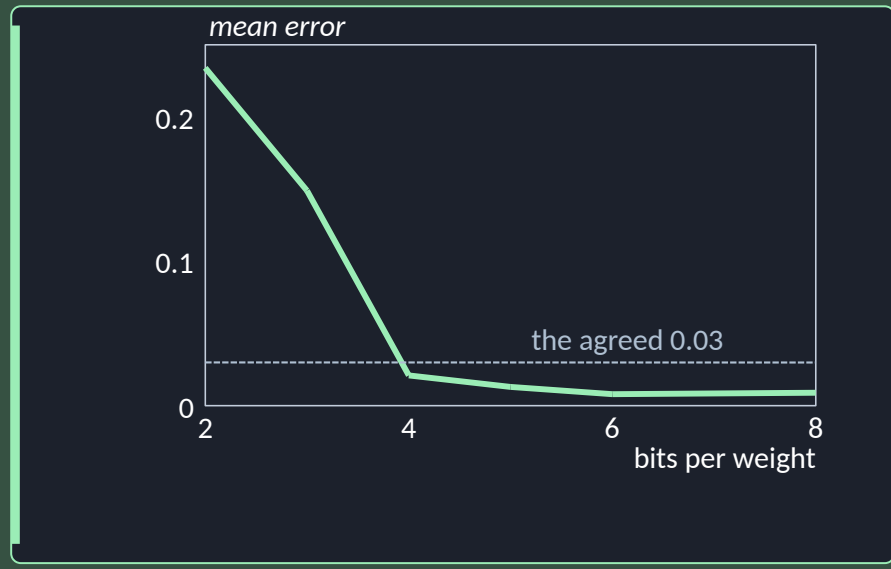
THE ROUNDING, AND WHERE IT BREAKS

$$w_b = \Delta \,\mathrm{round}\left(\frac{w}{\Delta}\right), \qquad \Delta = \frac{w_{\max} - w_{\min}}{2^b - 1}$$

w a weight, w_b the weight stored in b bits, Δ the step between the 2^b values kept

Ex_11.1 rounds its trained steerer to 8, 6, 5, 4, 3 and 2 bits: fine to four bits, an eighth of the storage, and broken at three - the cliff is sharp, so find it by measuring, not by assuming.

THE MEASURED CLIFF



0.009, 0.008, 0.013, 0.021 - then 0.149 and 0.234. Four bits ride; three crash.

Compiled for the Device

Compilation — translating the trained model into an engine built for one specific GPU: TensorRT fuses layers, chooses kernels and can drop to half precision, and on the Nano that is where the frame rate comes from. The engine is device-specific, so it is built on the car and timed there.

TRAIN OFF THE DEVICE, RUN ON IT

FROM A LAPTOP TO AN ENGINE



the engine is built for this one GPU, so convert on the car and time before and after

The Nano's tooling is years old, so training happens elsewhere and only the weights travel: the state dictionary, never the pickled model.

TALK · 2 MIN How would you decide, in your own words, how large a model your car can afford? Then, in the chain above, which arrow would you distrust first when the car behaves worse than the notebook?

WHAT THE CONVERSION BUYS

layer fusion	fewer trips through memory
half precision	the large win on this GPU
kernels	chosen for this one chip
the gain	measured on the car, not assumed
the order	pick the rate, then the model

Same model, same accuracy, several times the frame rate: the exercise's most satisfying measurement.

Advanced Topics

RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

topic	what it adds	read	where it makes a difference
obstacle avoidance	free or blocked: label the action, not the scene	NVIDIA JetBot	cluttered floors
dynamic quantisation	PyTorch's own, applied at run time	PyTorch	classifiers on a CPU
pruning	weights removed, not rounded	PyTorch	memory-bound models
the TensorRT stack	the compiler behind the on-car gain	torch2trt	any Jetson deployment
full autonomy stacks	mapping, planning, many sensors	ROS	beyond one camera

WHY THEY ARE LEFT OUT

Ex_11.1 follows one taped line with one camera, scored on drawn frames where the truth is known, so none of them is needed to do it. The failure signatures are taught where they arise, in the on-car notebook's own steps.

EXERCISE

Exercise 11.1

ONE STUDY NOTEBOOK, AND THE CAR

- **1 – 3** — the task, the drawn frames, the blind distance
- **4** — the classical detector, at four resolutions
- **5** — the CNN, trained along the line and trained deliberately
- **6** — the simulated lap; the weights in 8 down to 2 bits
- **7 – 9** — the report; then Ex11.1_on_the_car, in the lab



SETUP.md first: flash, 10 W mode, mount the camera once

THE QUESTION EACH PART ANSWERS

1 – 3	how far does the car drive blind?
4	when is classical enough?
5	what does deliberate data buy?
6	how few bits, how slow a loop?

REPORT QUESTION, TO CONCLUDE

The deliverables are two measured numbers: the loop rate of your car and the blind distance at your chosen speed - not the accuracy. What did you change to reach them, and what did each change cost?

SUMMARY

Key Takeaways

VISION POLICY

what it is	a CNN from a frame to an action
function	where the line is, a little ahead
train, tune	deliberate data; a reused backbone
vs a detector	survives glare and drift
used for	steering with one camera

LATENCY

what it is	the loop rate, and the blind $d = v/f$
function	decides what the car does, not sees
train, tune	bits and compilation, measured
vs accuracy	judged together, rate first
used for	choosing the model the car affords

Next: the physics comes back, as the car's own dynamics and energy.

Questions

1 – 2 · A NETWORK THAT STEERS

1 **How can a network steer the car from one camera frame, and when is a classical line detector enough?**

A small convolutional network takes one grey frame and returns one number, the position of the line a little ahead, learning its own features from frames whose true position is known. A classical detector that looks for bright pixels in a band of rows needs no training and is faster per frame. On clean frames both meet the agreed error; a patch of glare raises the detector's error more than tenfold, while the network, shown glare in training, keeps its error below 0.01. *UDL ch. 10; Ex_11.1 sections 4 – 5*

2 **What must the training frames show for the network to steer a drifting car, and why are few real frames enough?**

The network learns only what the frames show: the floor, the camera mount and the light it will drive in, and on purpose the positions the car reaches when it is already going wrong, labelled with the way back. Trained only along the line, the same network is seventeen times worse beside it. On the car a few hundred real frames are enough because a ResNet-18 pre-trained on ImageNet is reused, only its head is new, and its input is normalised as the backbone expects. *UDL ch. 9; Ex_11.1 section 5*

3 – 4 · THE NETWORK ON THE CAR

3 **Why is a steering network judged by the rate of the control loop as well as by its error?**

Between two decisions the car drives on its last command with no new information, over a blind distance of its speed divided by the decisions per second. On the simulated lap the car holds the line within 2 cm up to 10 cm blind and leaves the course at 17 cm, so a slightly more accurate network that halves the loop rate is usually a worse car. The whole loop is timed, from capture to actuation, because the network is only one of its stages. *Ex_11.1 sections 3, 6*

4 **How can a trained network be made fast enough for the car's small GPU, and what must be checked?**

Quantisation stores each weight in fewer bits, so the weights move faster through the small GPU's memory; compilation into an engine for that one GPU fuses layers, chooses kernels and drops to half precision. In the study notebook the steering network keeps its error down to four bits, an eighth of the storage, and fails at three, so the limit is found by measuring. The engine is specific to the device, so it is built and timed on the car itself. *Ex_11.1 section 6*

NEXT

L11.2 — Dynamics and Energy

The physics comes back: the same car as a dynamical system, and the cheapest lap as an optimisation with the grip limit built into a network.

BEFORE THE NEXT SESSION

read	UDL ch. 10
run	Ex_11.1's study notebook, 1 to 6
bring	your loop rate, and its blind distance
before the lab	SETUP.md, and a charged pack

In the lab: throttle limited in software before the first run, one hand on the kill switch, and work in pairs - one drives the software, one watches the car.